

PROJET DE CONCEPTION

Map Matching

*Algorithme pour le réalignement de traces GPS
sur des données cartographiques*

Timothée Bohé Thomas Gasq Ilyas Hanine

Coordinateur du projet : José PICAL
Département Signal et Statistiques
Laboratoire des Signaux et Systèmes (L2S)
CentraleSupélec

7 juin 2019

Résumé

Le *map matching* consiste à associer une trace GPS bruitée à un itinéraire cohérent sur un réseau routier. Ce rapport présente la conception et l'implémentation d'un algorithme probabiliste de réaligement fondé sur un *modèle de Markov caché* (HMM) résolu par l'algorithme de Viterbi. Après avoir motivé l'approche probabiliste face à deux approches naïves, nous détaillons la modélisation mathématique (matrices d'émission et de transition, estimation des paramètres σ et β), la chaîne de traitement des données cartographiques issues d'OpenStreetMap, puis les tests réalisés sur un secteur d'essai. Nous discutons enfin les limites de l'implémentation, principalement liées à la complexité algorithmique.

Table des matières

1	Introduction — Map Matching	1
1.1	Enjeux, défis, motivation	1
1.2	Présentation du problème	2
1.2.1	Première approche : approche naïve	3
1.2.2	Deuxième approche : projection sur le segment le plus proche	3
1.2.3	Troisième approche : approche probabiliste	4
2	Modélisation mathématique du matching	5
2.1	Principe	5
2.2	Matrice d'émission	6
2.3	Matrice de transition	7
2.4	Estimation des paramètres	7
2.4.1	Estimation de σ	7
2.4.2	Estimation de β	7
2.5	Algorithme de Viterbi	7
3	Manipulation des données	8
4	Tests et résultats	10
	Lexique — définitions importantes	11
	Références bibliographiques	11

1 Introduction — Map Matching

1.1 Enjeux, défis, motivation

La généralisation des systèmes GPS dans les dispositifs électroniques dédiés aux activités sportives permet d'acquérir simplement un grand nombre de traces GPS, qui peuvent constituer

une base de données exploitable pour de nombreuses applications. L'utilisation des traces GPS nécessite cependant de passer par une opération de *map matching*, qui consiste à faire correspondre les points GPS à leur position réelle sur la route.

Ce projet s'intègre dans le cadre d'un projet plus vaste lié au développement d'outils (algorithmes de routage spécifiques, analyse statistique des usages, etc.) pour les circulations douces (vélo, marche à pied), pour lesquelles le map matching est une brique essentielle. L'enjeu est donc de taille, étant donné que le map matching est l'aspect fondamental permettant potentiellement à d'autres projets d'exister.

Nous sommes trois étudiants intéressés par le traitement et la manipulation de données ainsi que par la conception d'algorithmes ; c'est donc naturellement que nous nous sommes intéressés à ce projet, qui nous a permis de monter en compétences dans les domaines mathématiques et informatiques.

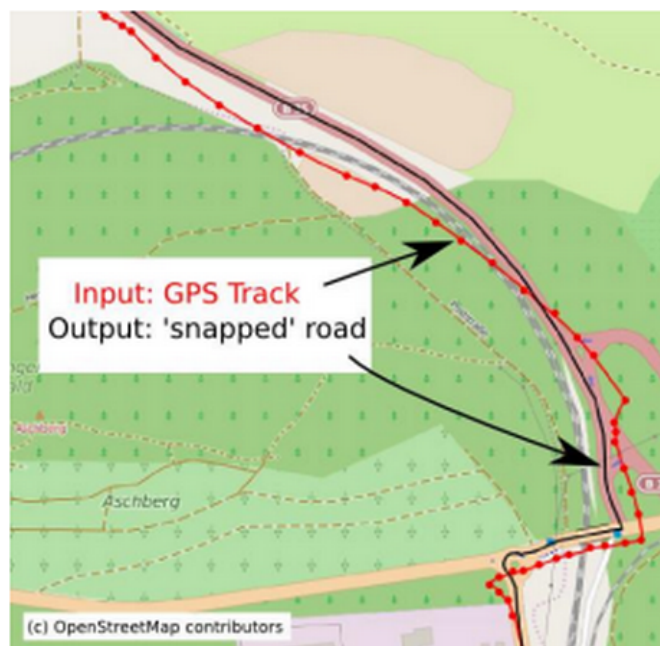


Figure 1 – Principe du map matching : une trace GPS bruitée (*input*) est réalignée sur le réseau routier réel (*output*).

1.2 Présentation du problème

Nous nous intéressons donc à la résolution d'un problème non trivial, le map matching : faire correspondre un relevé de points GPS à un chemin logique de routes. On dispose d'un ensemble de coordonnées GPS et l'on cherche à obtenir un chemin dans un graphe représentant les routes. Les routes sont elles-mêmes représentées par un ensemble de points appelés *nœuds*.

Plusieurs difficultés se présentent. Les points GPS sont entachés d'une certaine perturbation, ce qui impose de retrouver le chemin réel parmi les routes adjacentes. Les routes étant définies par une suite de nœuds, une première approche consisterait à associer chaque point GPS au nœud le plus proche.

1.2.1 Première approche : approche naïve

L'approche la plus directe associe chaque point GPS au nœud le plus proche (fig. 2).

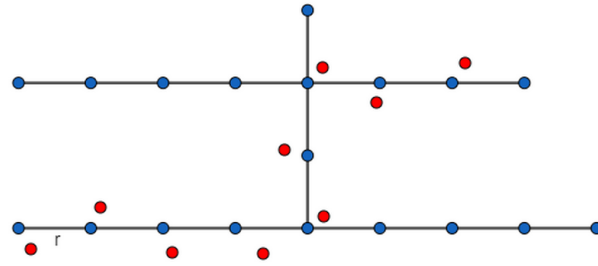


Figure 2 – Première approche : association au nœud le plus proche.

Mais les nœuds ne sont pas forcément régulièrement espacés et certains cas posent problème. Si les points GPS se trouvent autour d'un segment dont les extrémités sont éloignées, on obtient un mauvais résultat (fig. 3).

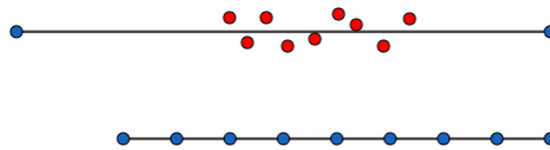


Figure 3 – Réalignement erroné produit par l'approche naïve.

1.2.2 Deuxième approche : projection sur le segment le plus proche

Une amélioration consiste à projeter chaque point GPS sur le segment de route le plus proche (fig. 4).

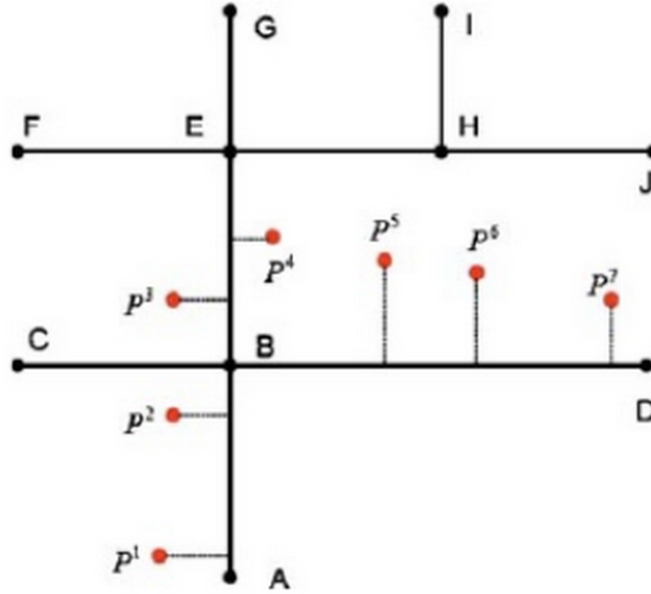


Figure 4 – Projection des points GPS sur les routes adjacentes.

Cependant, projeter chaque point sur la route la plus proche peut conduire à des incohérences. On voit sur la fig. 4 que le chemin passerait par $AB-BE-BD$ en effectuant un demi-tour sur BE . Il faut donc considérer plusieurs routes possibles pour chaque point et retenir le chemin le plus probable parmi toutes les possibilités.

1.2.3 Troisième approche : approche probabiliste

On adopte finalement un modèle probabiliste pour trouver le chemin le plus probable parmi les chemins possibles, en se fondant sur plusieurs critères :

- la distance entre un point GPS et son projeté sur la route ;
- la distance de graphe entre les points projetés ;
- la différence entre la distance à vol d'oiseau séparant deux points GPS consécutifs et la distance de graphe entre leurs projetés.

L'utilité des deux premiers critères est immédiate. Le troisième garantit le maintien de l'allure de la trace. Sur la fig. 5, le premier critère favorise légèrement la route du haut, tandis que le deuxième favorise fortement la route du bas ; sans le troisième critère, on retiendrait la route du bas, qui ne conserve pas l'allure de la trace.

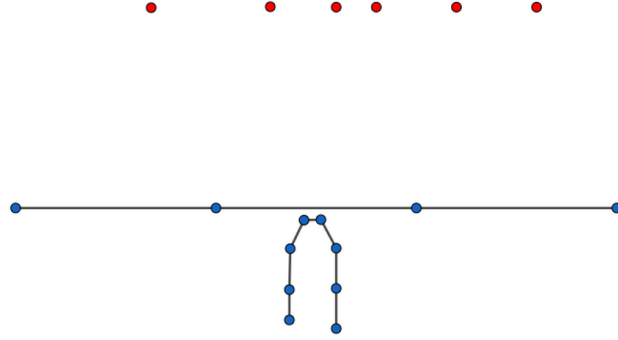


Figure 5 – Nécessité de comparer distance à vol d’oiseau et distance de graphe.

Plusieurs points se dégagent : il faut traiter la trace GPS dans son ensemble, considérer pour chaque point GPS plusieurs routes candidates (un nombre fixé de routes par point, ou bien les routes situées dans un rayon donné), puis calculer le chemin le plus probable à partir des critères précédents.

2 Modélisation mathématique du matching

Dans cette partie, nous précisons en quoi consiste mathématiquement le matching et quelle est la théorie probabiliste sous-jacente. Pour effectuer le matching, nous nous sommes appuyés sur les *modèles de Markov cachés*, ou *Hidden Markov Models* (HMM) [2].

2.1 Principe

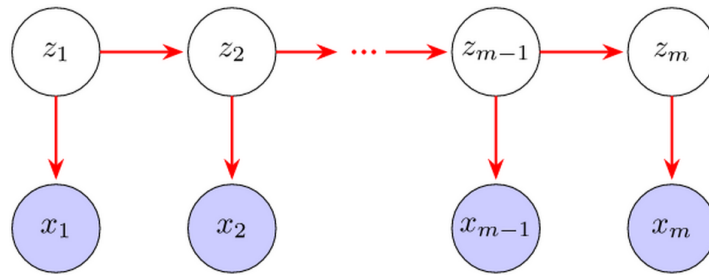


Figure 6 – Modélisation par un Hidden Markov Model.

Le principe est le suivant : un HMM est constitué d’états et de variables observables et inobservables. Le but est de retrouver l’état du système à chaque instant à partir des variables observables. On construit donc une maille (fig. 7) et, grâce à l’algorithme de Viterbi, on détermine le trajet qui maximise le produit des probabilités présentées plus bas.

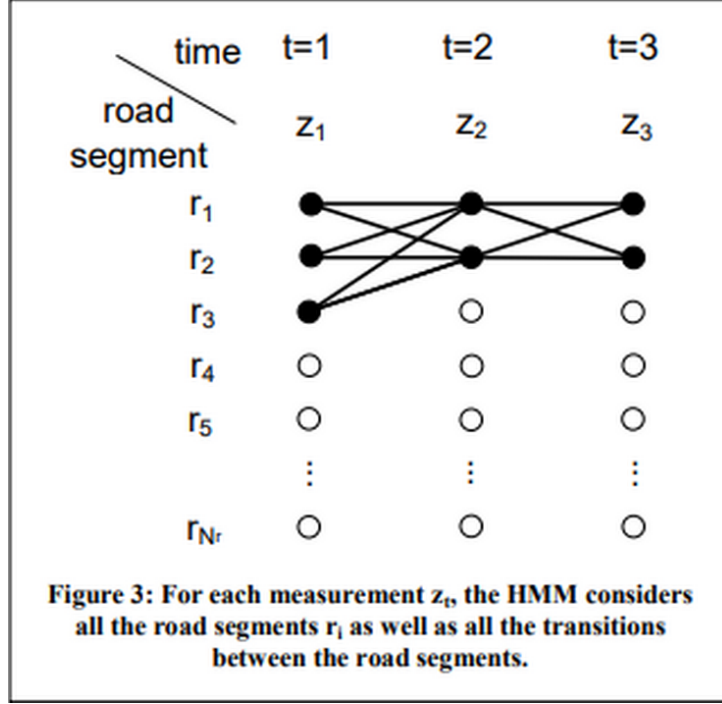


Figure 7 – Maille des états et des transitions.

Dans ce modèle, les états du processus sont les différentes routes susceptibles d’accueillir l’usager. Ce que l’on observe — la trace GPS — correspond à des observations bruitées de ces états, la variable cachée étant la position véritable sur la route, que l’on ne peut pas observer. Nous partons donc des observations pour tenter de retrouver les variables non observables et les états. Pour caractériser le processus, nous utilisons des matrices d’émission et de transition, décrites ci-dessous et associées à des probabilités.

2.2 Matrice d’émission

À chaque observation et chaque route candidate, on associe une *probabilité d’émission* : la probabilité que la trace ait été émise par un usager se trouvant sur une route donnée. Elle suit une loi normale et dépend essentiellement de la distance entre le point GPS z et son projeté x sur la route r :

$$p(z \mid r) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2} \left(\frac{\|z - x\|_{\text{gc}}}{\sigma}\right)^2\right), \quad (1)$$

où σ représente l’écart-type du bruit associé à la mesure des traces GPS et $\|\cdot\|_{\text{gc}}$ désigne la *great-circle distance*, c’est-à-dire la distance à vol d’oiseau [1]. Il s’agit de la probabilité conditionnelle d’avoir observé le point z à l’instant t sachant que l’usager se trouve sur la route r . Cette probabilité est intuitive : plus le point GPS est éloigné de la route, moins il a de chances d’avoir été émis depuis un point de cette route.

Toutes ces probabilités sont regroupées dans une matrice B , ensuite exploitée par l’algorithme de Viterbi. L’élément (i, j) désigne la probabilité que la route i émette le point GPS j ; on normalise de façon à obtenir une somme égale à 1 sur chaque ligne.

2.3 Matrice de transition

On s'intéresse ici à la probabilité de passer d'une route à une autre, notamment pour éviter un trajet absurde où l'utilisateur changerait continuellement de route (par exemple entre deux routes parallèles). On associe une probabilité exponentielle de transition :

$$p(d) = \frac{1}{\beta} \exp\left(-\frac{d}{\beta}\right), \quad (2)$$

où d est la différence entre, d'une part, la distance à vol d'oiseau séparant deux points GPS consécutifs et, d'autre part, la distance de graphe entre leurs projetés sur le réseau routier ; β est un paramètre caractéristique de cette différence [1].

Ces probabilités sont regroupées dans une matrice de transition A , où l'élément (i, j) désigne la probabilité de passer de la route i à la route j ; on normalise de nouveau à somme unité sur chaque ligne. Cette matrice est recalculée à chaque instant, puisque la probabilité de transition à l'instant t dépend de la position des points GPS aux instants t et $t + 1$.

2.4 Estimation des paramètres

2.4.1 Estimation de σ

Pour estimer σ , qui mesure la dispersion de la trace GPS, on utilise

$$\sigma = 1,4826 \cdot \text{median}\left(\|z - x^*\|_{\text{gc}}\right), \quad (3)$$

où x^* désigne le projeté de la trace GPS sur la route où se trouve réellement l'utilisateur [1]. Il faut donc l'estimer indépendamment du matching, à partir de traces de référence dont on connaît la vraie position.

2.4.2 Estimation de β

Pour estimer β , caractéristique de l'écart entre distance à vol d'oiseau et distance de graphe, on utilise

$$\beta = \frac{1}{\ln 2} \text{median}\left(\left| \|z_t - z_{t+1}\|_{\text{gc}} - \|x_t^* - x_{t+1}^*\|_{\text{route}} \right|\right), \quad (4)$$

où $\|\cdot\|_{\text{route}}$ désigne la distance de graphe entre deux projetés. Comme pour σ , l'estimation nécessite de connaître la vraie route et doit donc être réalisée indépendamment du matching.

2.5 Algorithme de Viterbi

L'algorithme de Viterbi détermine, dans la maille des états, le chemin qui maximise le produit des probabilités. Étant donnée une séquence d'émission $e_1, \dots, e_n$, il calcule le chemin le plus probable parmi les états cachés, notés s_l . Dans le cas du map matching, la séquence d'émission est l'ensemble des traces GPS et les états cachés sont les segments de route candidats.

L'algorithme calcule de manière itérative le chemin le plus probable atteignant l'état s_l à l'étape i :

$$p(e_1 \dots e_i, s_l) = \max_{s_k \in S} \left(p(e_1 \dots e_{i-1}, s_k) \cdot p(s_k; s_l) \cdot p(s_l; e_i) \right), \quad (5)$$

où :

- $p(s_k; s_l)$ est la probabilité de transition de l'état caché s_k vers l'état caché s_l (cf. eq. (2)) ;
- $p(s_l; e_i)$ est la probabilité d'émission de l'état caché s_l (cf. eq. (1)).

Le chemin le plus probable atteignant s_l à l'étape i est alors le chemin le plus probable atteignant s_m — l'état réalisant le maximum dans eq. (5) — suivi de la transition de s_m vers s_l . La phase de *backtrace* reconstitue ensuite l'itinéraire complet.

Algorithme 1 Viterbi appliqué au map matching

Entrée : HMM , séquence d'émission $e_1, \dots, e_n$

Sortie : $Viterbi_path, p_{\max}$

```

1:  $(Paths, Probas) \leftarrow \text{INITIALIZE}(e_1, HMM)$ 
2: for  $i \leftarrow 2$  to  $n$  do
3:   for all  $s_l \in S$  do
4:      $(p(s_l), s_m) \leftarrow \max_{s_k \in S} (p(e_1 \dots e_{i-1})_{s_k} \cdot p(s_k; s_l) \cdot p(s_l; e_i))$ 
5:      $\text{UPDATE\_PATH}(s_l, s_m, Paths, New\_Paths)$ 
6:      $\text{UPDATE\_PROBA}(s_l, Probas, New\_Probas)$ 
7:   end for
8:    $Paths \leftarrow New\_Paths$ 
9:    $Probas \leftarrow New\_Probas$ 
10: end for
11:  $(Viterbi\_path, p_{\max}) \leftarrow \text{MOST\_PROBABLE}(Paths, Probas)$ 
12: retourner  $Viterbi\_path, p_{\max}$ 

```

3 Manipulation des données

Avant la partie « matching » proprement dite, un travail de manipulation des données a dû être fourni. Rappelons que nous utilisons des données issues d'OpenStreetMap au format JSON, structurées comme une liste de dictionnaires de deux types : **way** et **node**. Un objet **way** regroupe des caractéristiques potentiellement utiles à des algorithmes plus poussés (sens de circulation, usage de la route, restrictions d'accès, etc.) ainsi que la liste des identifiants des nœuds qui le composent. Un objet **node** se présente sous la forme

$$\{ 'type': 'node', 'id': \dots, 'lat': \dots, 'lon': \dots \},$$

où **lat** et **lon** désignent respectivement la latitude et la longitude.

Une première familiarisation avec les données a consisté à les représenter, c'est-à-dire à afficher une carte. Cela a nécessité une transformation des coordonnées géographiques en coordonnées cartésiennes :

$$x = R \cos \theta \cos \varphi, \quad y = R \sin \theta \cos \varphi, \quad (6)$$

où R désigne le rayon de la Terre, θ la longitude et φ la colatitude. On obtient ainsi la carte du plateau du Moulon et de Corbeville (fig. 8).

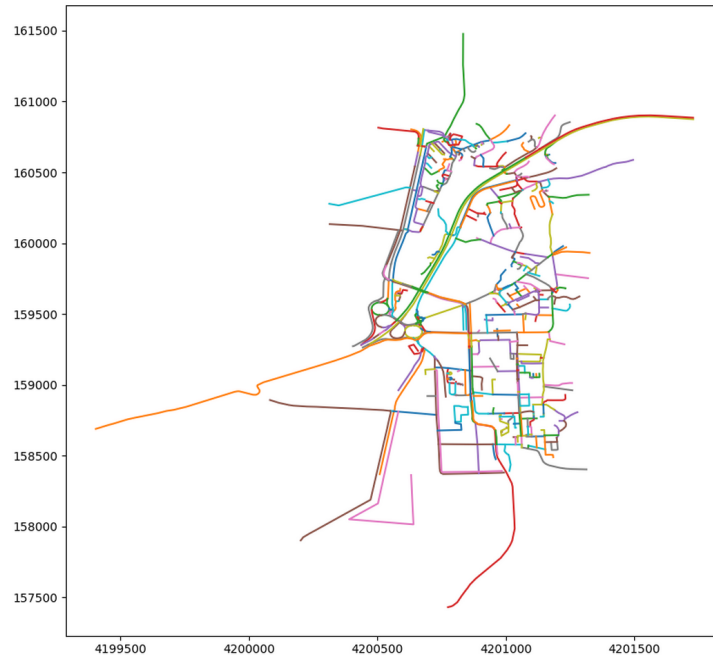


Figure 8 – Affichage des données cartographiques (plateau du Moulon, Corbeville).

Étant donné un point GPS, l’algorithme de map matching mis en œuvre nécessite de trouver les routes les plus proches de cette trace. L’approche consiste à parcourir l’ensemble des nœuds, à ne conserver que ceux situés dans un certain rayon de la trace GPS, puis à retrouver les routes auxquelles ils appartiennent. Cela ne suffit pas, comme l’illustre la fig. 9.

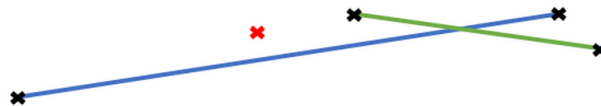


Figure 9 – Exemple de point GPS problématique. La croix rouge représente la trace GPS, les croix noires les nœuds des données ; la route 1 est en bleu et la route 2 en vert.

Le problème est que la route effectivement la plus proche est la route bleue, alors que l’approche précédente désigne la route verte. Cela vient du fait qu’une route n’est détectable que par ses nœuds : plus le maillage est fin et précis, moins ces approximations surviennent.

On introduit donc dans les données des *nœuds fictifs* entre les nœuds existants, avec un certain pas. Plus le pas est faible, plus le maillage est précis et plus la recherche de segments de proximité est fiable.

Encore faut-il pouvoir décrire l’ensemble des segments. À cette fin, la fonction `segments_node` prend en argument un fichier JSON et retourne une liste de couples de nœuds — structure naturelle pour décrire un segment. On identifie un segment par sa position dans cette liste.

La fonction `list_points` parcourt cette liste de couples de nœuds (les segments de route) et

crée, pour chaque nouveau nœud rencontré, une clé dont la valeur regroupe l'identifiant du nœud et la liste des identifiants de segments auxquels il appartient. À chaque étape, on crée également les nœuds fictifs, introduits de la même manière dans la valeur de retour. *In fine*, `list_points` retourne un dictionnaire dont les clés sont les identifiants des nœuds du réseau (fictifs et réels) et les valeurs la liste des identifiants de tous les segments auxquels chaque nœud appartient.

Enfin, `dist_pt_segments` retourne une liste de dictionnaires (un par trace GPS), dont les clés sont les identifiants des segments voisins et les valeurs une liste d'informations nécessaires (notamment les coordonnées des projetés sur les segments) à l'élaboration de la structure de données finale : un graphe où sont représentés tous les nœuds réels et tous les projetés des traces GPS. C'est ce graphe qui sera utilisé pour construire les matrices de transition et d'émission de l'algorithme de Viterbi.

4 Tests et résultats

On teste l'algorithme sur une route d'essai (trace GPS en rouge, graphe des routes en bleu). On échantillonne la route tous les mètres et l'on fixe $\sigma = \beta = 2$. Les capacités de l'algorithme ne permettent pas de traiter un graphe de trop grande taille. On obtient en un temps raisonnable (de l'ordre de la minute) le résultat du matching présenté fig. 11.

La complexité de l'algorithme ne permet de l'utiliser qu'à petite échelle, ce qui limite fortement la possibilité d'illustrer sa pertinence en fonction des paramètres pris en compte. La fig. 10 montre un exemple de trace inexploitable dans un temps raisonnable.

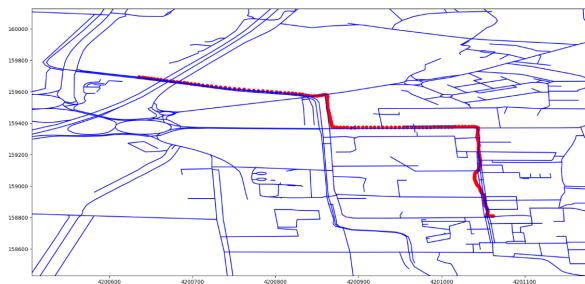


Figure 10 – Exemple de trace GPS intraitable dans un temps raisonnable.

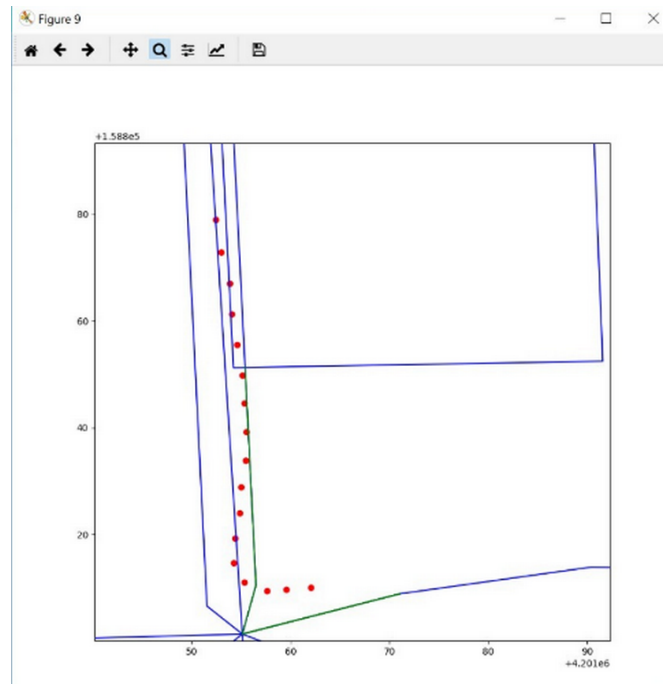


Figure 11 — Résultat du matching : trace GPS (rouge), réseau routier (bleu), itinéraire reconstitué (vert).

Lexique — définitions importantes

Nœud Point de la carte utilisé pour définir les routes et caractériser leurs intersections.

Route Liste de nœuds. Chaque route possède des caractéristiques propres, comme son type (*highway, street, etc.*).

Segment Deux nœuds consécutifs forment un segment. Une route est donc une succession de segments.

Point GPS Point du plan émis par une antenne GPS fictive située sur une route. Cette émission est bruitée, et c'est ce bruit qui pose problème dans le matching.

Trace GPS Ensemble de points GPS — par exemple la trace qu'émettrait un randonneur transmettant sa position toutes les minutes pendant une heure.

Distance de graphe Plus courte distance entre deux points situés sur le réseau routier (par exemple deux projetés de points GPS). Elle est calculée par l'algorithme de plus court chemin de Dijkstra ; c'est la distance qu'emprunterait un véhicule pour relier deux points.

Références

- [1] P. NEWSON et J. KRUMM, *Hidden Markov Map Matching Through Noise and Sparseness*, Microsoft Research, ACM GIS, 2009. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/map-matching-ACM-GIS-camera-ready.pdf>
- [2] *Map Matching @ Uber* (présentation des HMM appliqués au map matching), vidéo en ligne. <https://www.youtube.com/watch?v=ChtumoDfZXI>

- [3] M. PETIT et H. CHRISTIANSEN, *Un calcul de Viterbi pour un modèle de Markov caché contraint*, Cinquièmes Journées Francophones de Programmation par Contraintes, Orléans, juin 2009, p. 285–295 (hal-00390903).